

Jetson ↔ Mega 통신 프로토콜 변경 안내

제어 권한 이전 + 순회 사이클 관리 이전 — 2026-07-15 (v3) / STREW Jetson Robot 프로젝트

1. 이 문서의 목적 및 개정 이력

외부 팀 회의 결과로 로봇의 판단 권한과 순회 관리가 Jetson에서 Mega로 이전되기로 결정되었습니다. 이 문서(v3)는 그 전체 내용을 한 번에 정리한 최신 버전입니다. v1은 판단 권한 이전만 다뤘고, v2는 진행상황 보고(PROGRESS_UPDATE)와 타임아웃 주의사항을 추가했으며, 이번 v3에서는 1~4번 셀 전체 순회를 Mega가 자체 관리하는 것으로 범위가 확장되어 메시지 체계가 다시 정리되었습니다. 이전 버전 문서를 받으셨다면 이 v3로 대체해 주세요.

2. 변경 배경과 근거

기존 구조에서는 Jetson이 카메라/AI로 식물 상태를 판단하고 그 판단을 그대로 명령으로 만들어 Mega에 내려보냈으며, 셀 하나하나도 Jetson이 지정했습니다. 이번 변경으로 (1) REPLACE/OBSERVE/SKIP 판단, (2) 1~4번 셀을 어떤 순서로 언제 방문할지까지 전부 Mega가 자체 관리합니다. 이유는 다음과 같습니다.

- Jetson의 연산/통신 부하 감소 — 셀마다 지시를 받을 필요 없이 Mega가 한 번에 전체 순회를 처리합니다.
- 현장 대응성 — Mega가 물리 동작 도중 즉시 판단하고 대응할 수 있습니다.
- 역할 분담 재정비 — Jetson은 "순회 시작 트리거 + 비전 판단 제공 + AWS 중계"로, Mega는 "판단 + 순회 관리 + 실행"으로 책임이 재편됩니다.

주의: 기존 설계 원칙 "Mega는 명령을 받기 전엔 절대 스스로 움직이지 않는다"는 폐기됩니다. 이제 Mega가 동작 도중 먼저 Jetson에 말을 걸 수 있고(비전 확인 요청), 통신 방향이 양방향입니다.

3. Mega의 상위 동작 상태 (신규)

이번 v3에서 확정된 Mega의 상위 동작 상태 3가지입니다. 아래 4절의 메시지들은 이 상태 전환과 맞물려 동작합니다.

Mega 상태	의미	전환 조건
IDLE	Jetson의 명령을 기다리는 대기 상태	순회 완료 후, 또는 시작 시 기본값
RUN	1~4번 순회 실행 중	START_CYCLE 수신 시 IDLE → RUN
ERROR	내부 문제로 비상 정지	하드웨어/센서 이상 등 감지 시 즉시 전환 (RUN/IDLE 무관)

동작 개요: 1번부터 4번까지 순서대로 순회하며 각 셀에서 비전 확인 → 이상 있으면 즉시 그 자리에서 보식(REPLACE) 수행 → 다음 셀로 이동. 4번까지 전부 마치고 전체가 정상 처리되면 초기 위치(1번)로 복귀해 대기(IDLE)합니다. 검사 중간에 병해충이 발견되면 그 즉시 보식 작업을 하고 계속 순회를

이어갑니다.

4. 신규 메시지 프로토콜 상세 (7종)

전송 형식은 기존과 동일합니다 — 한 줄 JSON + 개행문자(\n), UTF-8 인코딩. 모든 메시지는 "type" 필드로 종류를 구분합니다. 이전 버전 문서에 있던 ASSIGN_TARGET(셀 하나씩 지정)은 이번 v3부터 사용 중단되었습니다 — START_CYCLE로 대체되었습니다.

type	방향	필드	설명
START_CYCLE	Jetson → Mega	(없음)	"순회 시작해"만 알려줌. 셀 지정 없음 — 1~4번 중 어디로 갈지는 Mega가 자체 관리. IDLE → RUN 전환. 한 순회당 1회.
REQUEST_VISION	Mega → Jetson	(없음)	순회 중 셀마다 필요할 때 보냄(위치 도착 직후, REPLACE 완료 후 검증 시 등). 한 순회에 여러 번 가능.
VISION_RESULT	Jetson → Mega	status (healthy/powdery_mildew/missing_plant/empty_cell)	REQUEST_VISION 응답. 날것의 비전 결과만 담음 — 판단은 Mega가 함.
PROGRESS_UPDATE	Mega → Jetson	target, state, progress(선택)	순회 중 진행상황 정보성 보고. 응답 불필요. Jetson은 AWS로 그대로 중계.
REPORT_RESULT	Mega → Jetson	target, execute_task, completion, success(선택)	셀 하나 처리 결과. 한 순회당 최대 4회(셀마다 1번).
CYCLE_COMPLETE	Mega → Jetson	(없음, 필요시 요약 필드 협의 가능)	전체 순회(1~4번) 완료 + 초기 위치 복귀 + IDLE 전환 신호. 한 순회당 1회. Jetson은 이걸 받아야 다음 START_CYCLE을 보냄.
ERROR	Mega → Jetson	reason(선택)	내부 문제로 비상 정지 시 즉시 전송. Jetson은 자동으로 다음 순회를 트리거하지 않음(9절 참고).

메시지 예시 (JSON)

START_CYCLE (Jetson → Mega):

```
{"type": "START_CYCLE"}
```

REQUEST_VISION (Mega → Jetson):

```
{"type": "REQUEST_VISION"}
```

VISION_RESULT (Jetson → Mega):

```
{"type": "VISION_RESULT", "status": "powdery_mildew"}
```

PROGRESS_UPDATE (Mega → Jetson):

```
{"type": "PROGRESS_UPDATE", "target": "cell_3", "state": "PICK_OLD_POT", "progress": 40}
```

REPORT_RESULT (Mega → Jetson):

```
{"type": "REPORT_RESULT", "target": "cell_3", "execute_task": "REPLACE", "completion": "COMPLETE", "success": true}
```

CYCLE_COMPLETE (Mega → Jetson):

```
{"type": "CYCLE_COMPLETE"}
```

ERROR (Mega → Jetson):

```
{"type": "ERROR", "reason": "gripper jam"}
```

인코딩 규칙: `json.dumps(payload, separators=(",", ":"), ensure_ascii=False) + "\n"` 을 UTF-8로 인코딩해 그대로 전송합니다. 개행(\n)까지 한 줄을 읽어 UTF-8 디코딩 후 JSON 파싱하면 됩니다 (ArduinoJson 등 사용 권장). 파싱 실패 시 그 줄은 무시하고 다음 줄을 기다리는 방어 코드를 권장합니다.

5. 통신 흐름 예시 (1~4번 순회 중 3번에서 REPLACE 발생)

```
Jetson --START_CYCLE-----> Mega (IDLE → RUN)
Mega: 1■ ■■ ■■
Mega --REQUEST_VISION-----> Jetson
Jetson --VISION_RESULT(status=healthy)-> Mega (1■: ■■ ■■)
Mega --REPORT_RESULT(cell_1, OBSERVE, COMPLETE)--> Jetson
Mega: 2■ ■ ■■ (■■ ■■ ■■, healthy ■■)
Mega: 3■ ■■ ■■
Mega --REQUEST_VISION-----> Jetson
Jetson --VISION_RESULT(status=powdery_mildew)--> Mega
Mega: ■■ ■■ → REPLACE, ■■ ■■ ■■ ■■
(■■ ■ REQUEST_VISION ■ ■ ■ → ■■ ■ ■■)
Mega --REPORT_RESULT(cell_3, REPLACE, COMPLETE, success=true)--> Jetson
Mega: 4■ ■ ■■ (healthy ■■) → ■■ ■■ ■■
Mega: ■■ ■■(1■) ■■
Mega --CYCLE_COMPLETE-----> Jetson (RUN → IDLE)
Jetson: ■■ START_CYCLE ■■■ ■■■■
```

6. Mega 펌웨어가 새로 구현해야 할 것 (체크리스트)

- ① IDLE/RUN/ERROR 상태 관리 — 위 3절의 상태 전환 규칙대로 내부 상태를 관리합니다.
- ② START_CYCLE 수신 시 순회 시작 — IDLE 상태에서만 반응하고, 1번 셀부터 순서대로 이동을 시작합니다.
- ③ 셀마다 REQUEST_VISION 전송 + 응답 대기 타임아웃 (필수, 9절 참고) — 응답이 일정 시간(권장 5~10초) 내에 오지 않으면 무한정 기다리지 말고 안전 정지 또는 재시도로 대응해야 합니다.
- ④ VISION_RESULT 수신 후 판단 로직 실행 — 아래 7절의 판단 규칙표를 그대로 C++ 로직으로 구현합니다.
- ⑤ 판단 결과에 따른 물리 동작 즉시 수행 — 병해충 발견 시 그 자리에서 바로 보식 작업 수행 후 다음 셀로 이동.
- ⑥ (선택) PROGRESS_UPDATE 전송 — 세부 단계가 바뀔 때마다 target/state/progress를 보내면 AWS에서 실시간 진행률을 볼 수 있습니다.
- ⑦ 셀마다 REPORT_RESULT 전송 — 한 셀 처리가 끝날 때마다(최대 4회) 결과를 보고합니다.
- ⑧ 4번 셀까지 끝나면 초기 위치 복귀 후 CYCLE_COMPLETE 전송 — 이 신호가 와야 Jetson이 다음 순회를 트리거할 수 있습니다.
- ⑨ 내부 문제 감지 시 즉시 ERROR 전송 + 안전 정지 — reason 필드에 원인을 담아 보내주시면 AWS 쪽에서 확인하기 좋습니다.

7. REPLACE/OBSERVE/SKIP 판단 규칙 (그대로 이식할 스펙)

기존 Jetson의 robot/planner.py 규칙을 그대로 옮긴 것입니다. VISION_RESULT의 status 값 기준으로 판단하며, 표에 없는 값은 전부 SKIP(기본값) 처리합니다.

status (VISION_RESULT로 수신)	execute_task (Mega가 결정)
healthy	OBSERVE
powdery_mildew	REPLACE
missing_plant	REPLACE
empty_cell / 그 외 인식 안 된 값	SKIP (기본값)

8. 알려진 미해결 사항 및 주의사항 — ①은 Mega 쪽 구현이 꼭 필요합니다

① [중요] REQUEST_VISION 응답 지연 시 데드락 위험. Jetson의 비전 처리가 멈추거나 지연되면 VISION_RESULT가 영영 안 올 수 있습니다. Mega 쪽에서 REQUEST_VISION 전송 후 응답 대기 시 타임아웃(권장 5~10초)을 반드시 걸어주시고, 타임아웃 시 안전 정지 · 재요청 중 어떤 방식으로 대응할지 함께 논의가 필요합니다. Jetson 쪽 보완은 저희가 별도 진행합니다.

② ERROR 이후 재시작 방법 미설계. Jetson은 ERROR를 받으면 안전을 위해 자동으로 다음 순회를 트리거하지 않도록 해렸습니다. 사람이 문제를 확인한 뒤 "다시 시작해도 좋다"고 알려줄 방법(예: 별도 RESET 메시지)이 아직 없습니다 — 함께 설계가 필요합니다.

③ 시리얼 쓰기 동시 접근 위험. Jetson 내부에서 두 스레드가 같은 시리얼 포트에 쓸 수 있어 드물게 바이트가 섞일 위험이 있습니다(Jetson 쪽에서 Lock으로 보완 예정). Mega 쪽에서도 JSON 파싱 실패 시 그냥 무시하고 다음 줄을 기다리는 방어 코드를 권장드립니다.

9. 요약

Jetson은 이제 "순회 시작 트리거(START_CYCLE) + 비전 결과 제공 + 진행상황/결과 AWS 중계"만 담당합니다. Mega는 1~4번 셀을 자체적으로 순회하며 셀마다 비전을 요청해 REPLACE/OBSERVE/SKIP을 스스로 판단 · 실행하고, 전체 순회가 끝나면 초기 위치로 복귀해 CYCLE_COMPLETE를 보고합니다. 메시지 7종의 정확한 필드는 4절, 판단 규칙은 7절을 참고해 구현해 주시면 됩니다. 8절의 타임아웃(①)과 재시작 설계(②)는 필수로 함께 논의 부탁드립니다, 궁금한 점은 언제든지 회신 부탁드립니다.